

EKSAMEN

PG3401

14 MAI 2025

Kandidatnr: 141

TASK 1, THEORY:

a) Bruksområder for programmeringsspråket C

Programmeringsspråket C ble utviklet i perioden 1969-1973 av Dennis Ritchie ved AT&T Bell Laboratories, opprinnelig for å implementere operativsystemet UNIX (Gookin, 2020, Wiley). C er et allsidig, effektivt og relativt maskinnært språk, noe som gjør det egnet for en lang rekke formål innen programvareutvikling. Språket brukes mye til systemprogrammering, for eksempel skriving av operativsystemkjerner og drivere, samt innebygde systemer (firmware for mikrokontrollere og elektronikk) (King, 2008, Prentice Hall). Videre benyttes C til å utvikle kompilatorer og tolker for andre språk, databasehåndteringssystemer, nettverksprogramvare og spillmotorer. Dets brede anvendelse illustreres ved at sentrale systemer som Windows-operativsystemet, Oracle-databasemotoren, versjonskontrollsystemet Git og Python-tolkeren alle er skrevet i C (Kernighan & Ritchie, 1988, Prentice Hall). At C både gir lavnivåkontroll over maskinvaren og kan kompileres for mange plattformer, gjør at språket fortsatt anses som et "portabelt assemblerspråk" som kan brukes til de fleste oppgaver.

b) Dennis Ritchie og hans bidrag til IT

Dennis MacAlistair Ritchie (1941-2011) var en amerikansk dataingeniør som er mest kjent for å ha utviklet programmeringsspråket C og (sammen med Ken Thompson) operativsystemet Unix (Salus, 1994, Prentice Hall). Ritchie tilbrakte mesteparten av sin karriere ved AT&T Bell Labs, hvor han i samarbeid med Thompson skapte Unix på slutten av 1960- og begynnelsen av 1970-tallet. Disse banebrytende prosjektene la grunnlaget for moderne informatikk: Unix introduserte en rekke konseptuelle innovasjoner og har hatt enorm innflytelse på IT-verdenen (Moody, 2001, Penguin Books), mens C-språket muliggjorde portabel systemprogramvare. C og dets etterfølger C++ er fortsatt blant de mest brukte språkene for utvikling av applikasjoner, spill og operativsystemer, og har hatt stor innflytelse på nyere programmeringsspråk (Sebesta, 2011, Addison-Wesley). For sine bidrag mottok Dennis Ritchie blant annet Turing-prisen i 1983, regnet som "Nobelprisen" i informatikk (Association for Computing Machinery, 1983).

c) Eksempler på Linux-distribusjoner og deres bakgrunn

Linux finnes i mange varianter, kalt distribusjoner, som hver har ulik filosofi, målgruppe eller bruksområde. Under nevnes fem forskjellige Linux-distribusjoner, med bakgrunn og formål:

- **Debian:** Debian (grunnlagt 1993 av Ian Murdock) er en fri, community-drevet Linux-distribusjon. Den er en av de eldste fortsatt aktive distribusjonene og har et sterkt fokus på stabilitet og fri programvare (Debian Project, 2024, [debian.org](https://www.debian.org)). Debian-prosjektet koordineres av frivillige utviklere, og distribusjonen utgjør grunnlaget for en rekke andre Linux-OS. Debians konservative utgivelsesmodell prioriterer grundig testing fremfor hyppige oppdateringer.
- **Ubuntu:** Ubuntu (første utgivelse i 2004) er en populær Linux-distribusjon basert på Debian. Den utvikles av selskapet Canonical og legger vekt på brukervennlighet og regelmessige utgivelser (Canonical, 2024, [ubuntu.com](https://www.ubuntu.com)). Ubuntu kommer med et brukervennlig grafisk grensesnitt og bred maskinvarestøtte, noe som har gjort den til et av de mest utbredte Linux-systemene for nye brukere.
- **Fedora:** Fedora (lansert 2003) er en community-drevet distribusjon sponset av Red Hat. Fedora oppsto som en fortsettelse av det opprinnelige Red Hat Linux-prosjektet, og har som mål å ligge i front av utviklingen med de nyeste versjonene av programvare (Red Hat, 2024, getfedora.org).
- **Red Hat Enterprise Linux (RHEL):** RHEL (lansert 2003) er Red Hats kommersielle Linux-distribusjon rettet mot bedrifter. Den kjennetegnes av lang supportsyklus, høy stabilitet og profesjonell brukerstøtte (Red Hat, 2024, [redhat.com](https://www.redhat.com)). RHEL bygger på teknologi testet i Fedora og tilbyr forutsigbarhet og kompatibilitet med enterprise-programvare.
- **Arch Linux:** Arch Linux (lansert 2002) er en minimalistisk distribusjon med rolling release-modell. Arch leveres som et grunnsystem der brukeren selv bygger opp sitt miljø (Arch Linux Team, 2024, [archlinux.org](https://www.archlinux.org)). Filosofien "Keep It Simple, Stupid" gir brukeren full kontroll og tilgang til oppdatert programvare.
- **Kali Linux:** Kali Linux (lansert 2013) er en spesialisert Debian-basert distribusjon for cybersikkerhet. Den vedlikeholdes av Offensive Security og leveres med verktøy for digital etterforskning og penetrasjonstesting (Offensive Security, 2024, kali.org).

Referanser

Arch Linux Team. (2024). *Arch Linux*. Hentet fra <https://archlinux.org>

Association for Computing Machinery. (1983). *Turing Award: Dennis Ritchie*. Hentet fra <https://amturing.acm.org/>

Canonical. (2024). *Ubuntu*. Hentet fra <https://ubuntu.com>

Debian Project. (2024). *Debian*. Hentet fra <https://debian.org>

Gookin, D. (2020). *C Programming For Dummies* (2nd ed.). Wiley.

Kernighan, B. W., & Ritchie, D. M. (1988). *The C Programming Language* (2nd ed.). Prentice Hall.

King, K. N. (2008). *C Programming: A Modern Approach* (2nd ed.). W. W. Norton & Company.

Moody, G. (2001). *Rebel Code: Linux and the Open Source Revolution*. Penguin Books.

Offensive Security. (2024). *Kali Linux*. Hentet fra <https://kali.org>

Red Hat. (2024). *Fedora Project og Red Hat Enterprise Linux*. Hentet fra <https://getfedora.org> og <https://redhat.com>

Salus, P. H. (1994). *A Quarter Century of UNIX*. Addison-Wesley.

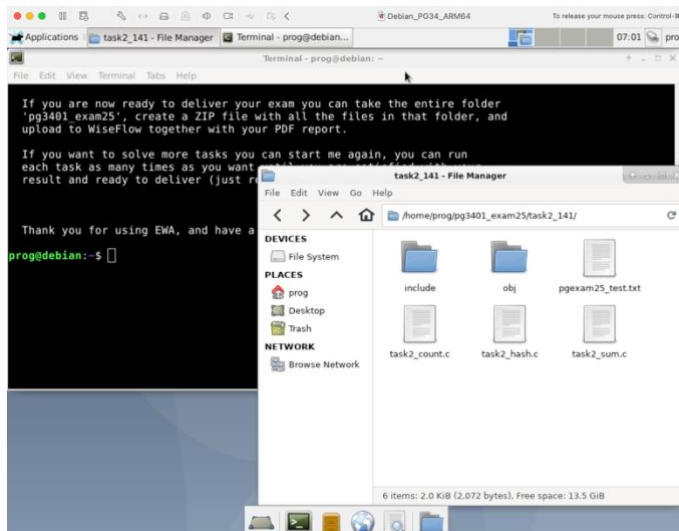
Sebesta, R. W. (2011). *Concepts of Programming Languages* (10th ed.). Addison-Wesley.

TASK 2, FILE MANAGEMENT AND FUNCTIONS:

I denne oppgaven har jeg laget et C-program som leser en tekstfil, kjører tre analyser (hash, bokstavtelling, filstørrelse og bokstavsum), og skriver resultatet til en binærfil.

Jeg brukte de tre kildefilene som EWA-verktøyet opprettet, uten å endre kildefilene, men laget headerfiler som matcher funksjonene. I main.c ble metadata-strukturen definert og fylt ut steg for steg ved å kalle funksjonene. Jeg la inn feilhåndtering (sjekk av NULL-pekere, filhåndtak, return-verdier) og inkluderte en egen debugger (pgdbglog) for å logge statuslinjer gjennom programmet.

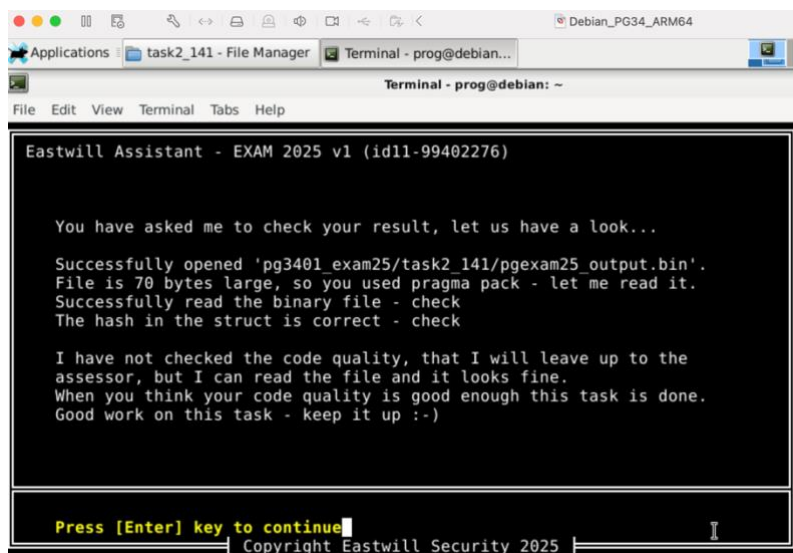
Jeg kjørte valgrind på det ferdige programmet og lagret rapporten som valgrind_report.txt i prosjektmappen. Rapporten viser ingen minnelekkasjer eller feil, og bekrefter at programmet håndterer minne og ressursfrigjøring korrekt.



Figur 1: Bilde av filstruktur og kildefilene med EWA

```
prog@debian:~/pg3401_exam25/task2_141$ make
gcc -c -o obj/task2_count.o task2_count.c -O2 -Iinclude -include stdio.h
gcc -c -o obj/task2_sum.o task2_sum.c -O2 -Iinclude -include stdio.h
gcc -c -o obj/task2_hash.o task2_hash.c -O2 -Iinclude -include stdio.h
gcc -c -o obj/main.o main.c -O2 -Iinclude -include stdio.h
gcc -c -o obj/pgdbglog.o pgdbglog.c -O2 -Iinclude -include stdio.h
gcc -o main obj/task2_count.o obj/task2_sum.o obj/task2_hash.o obj/main.o obj/pgdbglog.o -O2 -Iinclude -include stdio.h
prog@debian:~/pg3401_exam25/task2_141$ ./main
Metadata skrevet til pgexam25_output.bin
prog@debian:~/pg3401_exam25/task2_141$
```

Figur 2: Bilde av programmet som kjører og genererer binærfilen pgexam25_output.bin



Figur 3: Bilde av EWA som verifiserer binærfilen

TASK 3, LIST HANDLING:

I denne oppgaven har jeg utviklet et flyreservasjonssystem, som bruker en dobbeltlenket liste for å holde oversikt over fly og tilhørende passasjerer. Systemet lar brukeren legge til fly og passasjerer, søke, endre seter, og slette både passasjerer og fly.

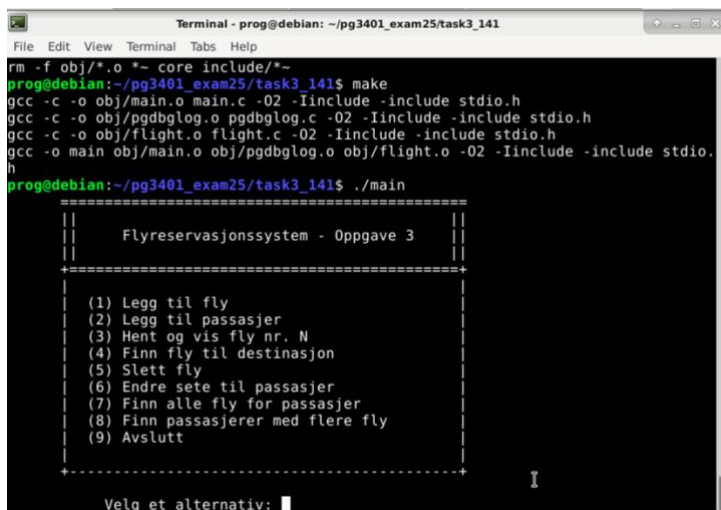
Jeg unngikk bruken av `scanf` ved å bruke `fgets` kombinert med `strcspn` for å fjerne linjeskift, for å gi tryggere innlesing av brukerinnt.

Også her inkluderte jeg en egen debugger (`pgdbglog`) for å logge statuslinjer gjennom programmet.

Jeg har brukt verktøyet Valgrind, som viser i vedlagt rapport at det ikke er noen minnelekkasjer eller feil.

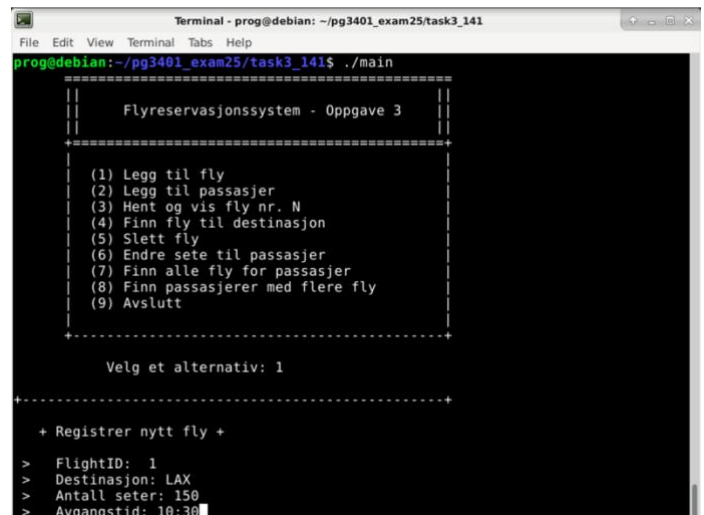
For å sikre korrekt og robust input fra brukeren har jeg implementert egne valideringsfunksjoner i C for å kontrollere at dataene som legges inn følger forventet format. Funksjonen `isDigitsOnly()` kontrollerer at felt som kun skal inneholde tall, som f.eks. *flightID*, *antall seter*, *avgangstid*, *setenummer* og *alder*, ikke inneholder bokstaver eller spesialtegn. Tilsvarende brukes `isLettersOnly()` for å sikre at felter som *navn* kun består av bokstaver. Ved ugyldig input vises en feilmelding, og registreringen avbrytes for å forhindre feil i datastrukturen. Dette forbedrer både brukervennlighet og datasikkerhet i programmet.

Til slutt la jeg inn en del timer med innsats for å gjøre programmet visuelt bedre å se på, og bruke.



```
Terminal - prog@debian: ~/pg3401_exam25/task3_141
File Edit View Terminal Tabs Help
rm -f obj/*.o *-core include/*~
prog@debian:~/pg3401_exam25/task3_141$ make
gcc -c -o obj/main.o main.c -O2 -Iinclude -include stdio.h
gcc -c -o obj/pgdbglog.o pgdbglog.c -O2 -Iinclude -include stdio.h
gcc -c -o obj/flight.o flight.c -O2 -Iinclude -include stdio.h
gcc -o main obj/main.o obj/pgdbglog.o obj/flight.o -O2 -Iinclude -include stdio.h
prog@debian:~/pg3401_exam25/task3_141$ ./main
=====
||
|| Flyreservasjonssystem - Oppgave 3 ||
||
+-----+
|
| (1) Legg til fly
| (2) Legg til passasjer
| (3) Hent og vis fly nr. N
| (4) Finn fly til destinasjon
| (5) Slett fly
| (6) Endre sete til passasjer
| (7) Finn alle fly for passasjer
| (8) Finn passasjerer med flere fly
| (9) Avslutt
|
+-----+
Velg et alternativ: |
```

Figur 4: Bildet viser menyen til programmet.



```
Terminal - prog@debian: ~/pg3401_exam25/task3_141
File Edit View Terminal Tabs Help
prog@debian:~/pg3401_exam25/task3_141$ ./main
=====
||
|| Flyreservasjonssystem - Oppgave 3 ||
||
+-----+
|
| (1) Legg til fly
| (2) Legg til passasjer
| (3) Hent og vis fly nr. N
| (4) Finn fly til destinasjon
| (5) Slett fly
| (6) Endre sete til passasjer
| (7) Finn alle fly for passasjer
| (8) Finn passasjerer med flere fly
| (9) Avslutt
|
+-----+
Velg et alternativ: 1
+-----+
+ Registrer nytt fly +
> FlightID: 1
> Destinasjon: LAX
> Antall seter: 150
> Avgangstid: 10:30
```

Figur 5: Bildet viser programmet som kjører en av oppgavene.

TASK 4. THREADS:

Sjekkliste for PART I (Task 4)

Makefile for bygging

Jeg har endret min makefile til å ta med nødvendige flagg (-lpthread og -lrt).

Ingen globale variabler

Alle tidligere globale variabler er fjernet og erstattet med en SharedData-struktur. Denne strukturen opprettes lokalt i main() og sendes som parameter til begge trådene.

Filnavn som kommando-linjeparameter

Jeg har fjernet det hardkodede filnavnet i thread_A. I stedet tar programmet inn filnavnet som et argument fra kommandolinjen (argv[1]), som lagres i SharedData og brukes når filen åpnes.

Eksplisitt initialisering av mutex og semaforer

Programmet bruker pthread_mutex_init() og sem_init() for å eksplisitt initialisere mutex og semaforer i main(). Alle ressurser ryddes opp med pthread_mutex_destroy() og sem_destroy() når programmet avsluttes.

Feilbeskrivelse:

EWA-verktøyet gir meldingen:

- Code not changed to using semaphores

Selv om programmet er oppdatert med semaforer og bygger og kjører som forventet.

Hva jeg har gjort for å rette opp feilen:

- Jeg har fjernet all bruk av pthread-condition-variabler (pthread_cond_*) og mutex-koding som tidligere ble brukt til synkronisering mellom trådene.
- Jeg har i stedet lagt til to POSIX-semaforer (sem_t sem_full, sem_t sem_empty) i SharedData strukturen.
- Jeg har initialisert semaforene eksplisitt i main() med sem_init().
- I trådene (thread_A og thread_B) bruker jeg sem_wait() og sem_post() for å styre tilgangen til bufferen.
- Jeg har brukt grep-kommandoene i terminal for å sjekke at det ikke lenger finnes pthread_mutex_* og pthread_cond_* kall i koden (bortsett fra at mutex fortsatt brukes til kritiske seksjoner).
- Jeg har testet programmet manuelt og sett at det kjører og produserer utdata uten feil.

```
prog@debian:~/pg3401_exam25/task4_141$ grep -nE "pthread_cond" task4_threads.*
prog@debian:~/pg3401_exam25/task4_141$
```

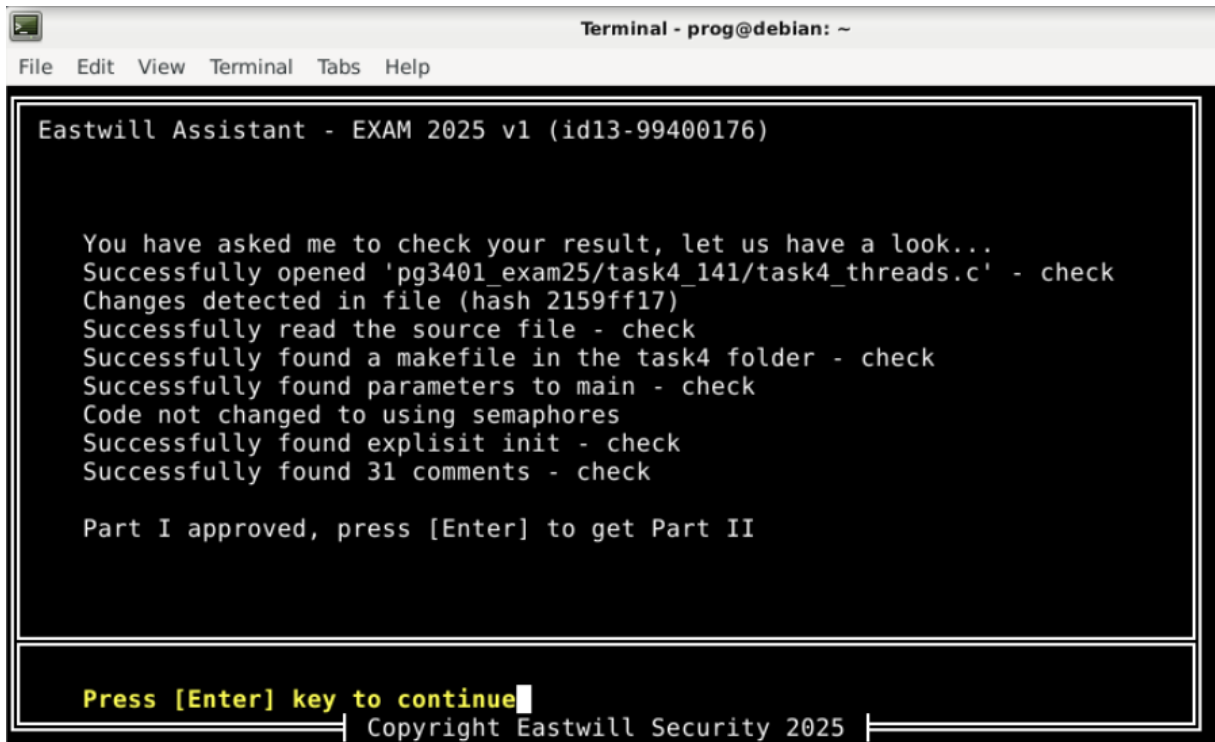
Figur 6: Bilde viser grep-kommando og ingen forekomst av pthread_cond i koden.

Hva jeg tror kan være årsaken til problemet:

EWA-verktøyet ser trolig etter at all synkronisering er flyttet fra `pthread_mutex_*` og `pthread_cond_*` over til kun semaforer.

Selv om jeg bruker semaforer for å regulere flyten mellom trådene, bruker jeg fortsatt `pthread_mutex_lock()` og `pthread_mutex_unlock()` for å beskytte selve dataoppdateringene inne i de kritiske seksjonene.

Det kan være at EWA forventer at jeg også fjerner mutex helt, og at semaforene alene skal beskytte både synkronisering og datasikkerhet – noe jeg har forstått ikke er like trygt i praksis, men som kanskje er intensjonen i oppgaveteksten?



```
Terminal - prog@debian: ~
File Edit View Terminal Tabs Help

Eastwill Assistant - EXAM 2025 v1 (id13-99400176)

You have asked me to check your result, let us have a look...
Successfully opened 'pg3401_exam25/task4_141/task4_threads.c' - check
Changes detected in file (hash 2159ff17)
Successfully read the source file - check
Successfully found a makefile in the task4 folder - check
Successfully found parameters to main - check
Code not changed to using semaphores
Successfully found explicit init - check
Successfully found 31 comments - check

Part I approved, press [Enter] to get Part II

Press [Enter] key to continue
Copyright Eastwill Security 2025
```

Figur 7: Bilde viser EWA etter fullførelsen av part 1.

I Part 2 fjernet jeg koden som teller byte-frekvens og implementerte i stedet beregning av DJB2-hash og TEA-kryptering av filen. Dette krevde at jeg fjernet for-løkken i `thread_B` og i stedet leste filen direkte inn for hashing og kryptering. Jeg beholdt `thread_A` tom, ettersom oppgaven ikke lenger krevde delt buffer.

Programmet genererte filene `task4_pg2265.hash` og `task4_pg2265.enc`. Hash-filen inneholder DJB2-hashverdien, og den krypterte filen viser binært, ulesbart innhold i hexdump. Alt ser ut til å virke som spesifisert i oppgaven.

Videre støtte jeg på et problem relatert til filnavnet for hash-funksjonen. EWA leverte en fil som het `dbj2.c`. (Skal være `djb2`).

På grunn av denne feilstavingen oppstod det problemer under bygging med `make`.

Jeg feilsøkte `makefile` og `dbj2.c` filen, og oppdaget heldigvis feilen raskt.

Feilen ble løst ved å endre filnavnet til `djb2` (Som er den korrekte måten å stave det på).

Om dette var et lurt triks fra sensor for å teste oppmerksomheten vår, må jeg si, well played!



Figur 8: Skjerm bilde av fil generert av EWA

Task 5. Network:

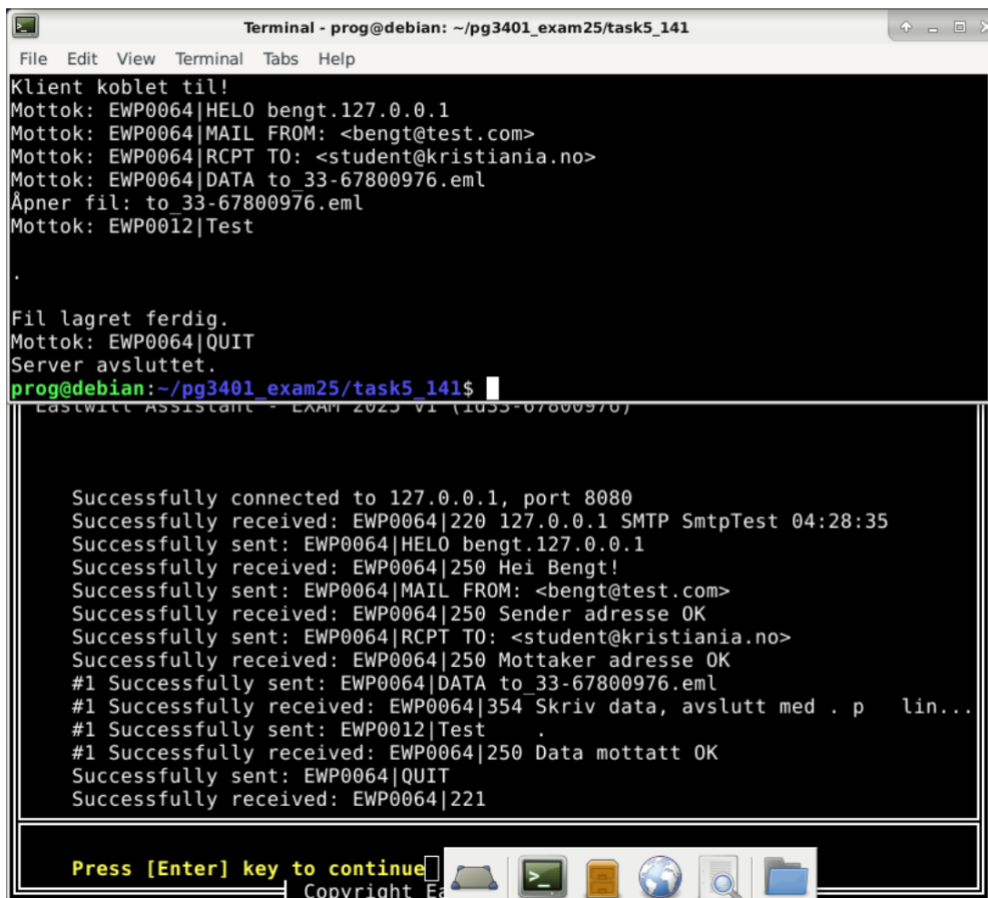
I denne oppgaven implementerte jeg en enkel SMTP-server, basert på de forhåndsdefinerte protokollene i headerfilen jeg fikk av EWA.

Serveren mottar kommandoene HELO, MAIL FROM, RCPT TO, DATA og QUIT, og svarer med passende statuskoder.

Hovedfokuset var å håndtere DATA-kommandoen korrekt, og skrive innhold til en .eml fil, og sikre filnavnsvalidering uten å kjenne filnavnet på forhånd.

Underveis støtte jeg på noen utfordringer:

- Ugyldige statuskoder, da jeg la feil koder i feil SERVERREPLY, og programmet ville ikke gå videre.
- Filnavnsavvisning; Jeg hadde problemer med at funksjonen isValidFilename() aksepterte originalt kun <.> og <_>, noe som ledet til at datafilen ble avvist, og programmet gikk ikke videredere. Dette måtte jeg endre til å også godkjenne <->, og en ekstra sjekk for .eml endelse. Da ble datafilen opprettet korrekt.
- Videre ble de opprettede filene lagret, men uten innhold. Jeg oppdaget at feilen lå i rekkefølgen i hovedløkken for dataMode, som gjorde at dataMode ble avsluttet før innholdet ble skrevet.
- Jeg fikk feilmeldingen «double free» i tcache. Etter nøye gjennomgang av koden så jeg at jeg hadde skrevet fclose(file) to ganger; 1 i dataMode, og 1 etter kommandoen «QUIT». Jeg løste dette ved å legge til koden «file = NULL» etter fillukkingen i dataMode løkken.



```
Terminal - prog@debian: ~/pg3401_exam25/task5_141
File Edit View Terminal Tabs Help
Klient koblet til!
Mottok: EWP0064|HELO bengt.127.0.0.1
Mottok: EWP0064|MAIL FROM: <bengt@test.com>
Mottok: EWP0064|RCPT TO: <student@kristiania.no>
Mottok: EWP0064|DATA to_33-67800976.eml
Åpner fil: to_33-67800976.eml
Mottok: EWP0012|Test
.
Fil lagret ferdig.
Mottok: EWP0064|QUIT
Server avsluttet.
prog@debian:~/pg3401_exam25/task5_141$

LastWill Assistant - EXAM 2023 V1 (1033-07800976)

Successfully connected to 127.0.0.1, port 8080
Successfully received: EWP0064|220 127.0.0.1 SMTP Smtptest 04:28:35
Successfully sent: EWP0064|HELO bengt.127.0.0.1
Successfully received: EWP0064|250 Hei Bengt!
Successfully sent: EWP0064|MAIL FROM: <bengt@test.com>
Successfully received: EWP0064|250 Sender adresse OK
Successfully sent: EWP0064|RCPT TO: <student@kristiania.no>
Successfully received: EWP0064|250 Mottaker adresse OK
#1 Successfully sent: EWP0064|DATA to_33-67800976.eml
#1 Successfully received: EWP0064|354 Skriv data, avslutt med . p lin...
#1 Successfully sent: EWP0012|Test
#1 Successfully received: EWP0064|250 Data mottatt OK
Successfully sent: EWP0064|QUIT
Successfully received: EWP0064|221

Press [Enter] key to continue
Copyright E
```

Figur 9: Skjerm bilde av programmet som kjører og mottar kommando og data fra EWA. EWA sender kommandoer og mottar statuskoder fra serverprogrammet.

TASK 6. PROBLEM SOLVING:

I denne oppgaven skulle jeg implementere et klientprogram som kan koble seg til en server, motta en kryptert fil, gjette TEA-nøkkel bestående av fire like bytes, og deretter dekryptere og lagre resultatet i en tekstfil med lesbar engelsk tekst bestående av 7-bit ASCII tegn.

Klienten klarer å:

- Klienten kobler seg korrekt til serveren
- Data mottas og skrives til buffer
- Gjette nøkkelen korrekt i enkelte tilfeller.
- Nøkkelen gjettes ved å dekryptere noen blokker og sjekke om resultatet består av lesbare ASCII-tegn.
- Når en nøkkel er funnet, forsøker programmet å dekryptere hele innholdet og skrive resultatet til en fil

Hva som ikke fungerer:

Til tross for at en nøkkel tilsynelatende blir funnet (f.eks. 0F0F0F0F eller 03030303), inneholder den dekrypterte filen ikke lesbar engelsk tekst, men i stedet tegnsekvenser med mye binær og uleselig data.

Loggen viser at jeg mottar encrypted bytes fra EWA, men at programmet dekrypterer bare 8 bytes.

Jeg tror dermed at problemet ligger i guess_key() funksjonen og at den avslutter etter bare én god blokk, noe som bare kan være en tilfeldighet. Derfor valideres feil nøkkel, og write_decrypted_blocks() finner ikke flere gyldige blokker.

Hva jeg har forsøkt for å løse problemet:

Kontrollert og forbedret guess_key() funksjonen:

- La til streng sjekk for at alle dekrypterte bytes er mellom ASCII 32 og 126.
- Begrenset validering til noen få blokker, for å effektivisere gjetting.
- Gikk gjennom alle 256 mulige byteverdier som kandidat til TEA-nøkkelen.

Korrigerte tea_decrypt()- implementasjonen:

- Dobbeltsjekket algoritmen mot offisiell TEA-spesifikasjon.
- Riktig rekkefølge og bitmanipulering ble gjennomgått og bekreftet.

Testet filutskrift med både binært og tekstlig fokus:

- Implementerte funksjon for å skrive ut dekryptert innhold som tekst **kun hvis alle** bytes var lesbare ASCII.
- Brukte hexdump -C for å manuelt sjekke innholdet og finne mønstre.

Kontrollert hele datastrømmen:

- Antall mottatte bytes ble logget og validert.
- Testet filskriving med fwrite(v, sizeof(unsigned int), 2, fp) og alternativt fwrite(p, 1, BLOCK_SIZE, fp) for å sjekke forskjeller.
- Prøvde å skrive kun de blokker som ble validert som tekst.

Undersøkte mulige feilkilder:

- Feil bitrekkefølge eller endianhet i systemet.
- Serveren sender kanskje padding eller data som ikke er kryptert med en nøkkel som oppfyller antakelsene våre.
- Nøkkelen gjettes feil fordi guess_key validerer blokker som tilfeldigvis ser "gyldige" ut.

Konklusjon:

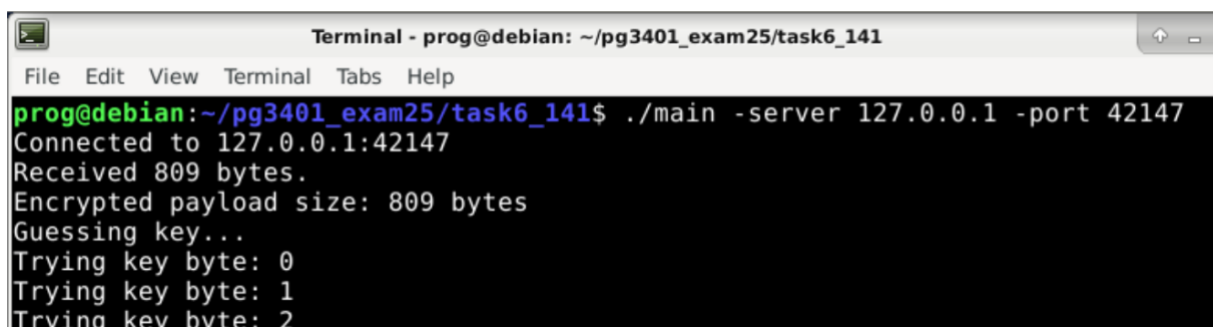
Programmet klarer å motta data, forsøke å gjette en nøkkel og skrive ut dekryptert innhold. Til tross for flere forsøk og forbedringer, forblir innholdet i filen uleselig. Det indikerer at det fortsatt er en feil i hvordan nøkkelen gjettes, hvordan data dekrypteres, eller hvordan validering av gyldig tekst gjøres.

Hva jeg ville fokusert på videre:

- Dypere analyse av de mottatte dataene og dekryptert innhold.
- Loggføring av hele dekrypteringsprosessen for sammenligning mellom nøkkel og innhold.
- Manuell test av alle 256 mulige nøkler for å se hva som faktisk produserer lesbart resultat.

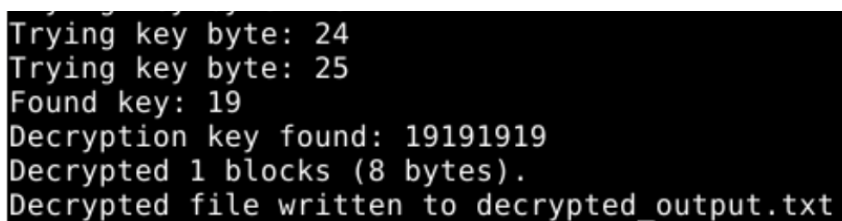
På grunn av begrenset tid prioriteres nå ferdigstilling av rapport og øvrige oppgaver. Likevel er mye av koden funksjonell og jeg mener den viser forståelse for nettverksprogrammering, binær databehandling og blokkbasert kryptering i C.

For å sikre at programmet ikke har minnelekkasjer eller ugyldige pekeroperasjoner, ble valgrind brukt. Valgrind-rapporten viser ingen minnelekkasjer eller feil.

A terminal window titled "Terminal - prog@debian: ~/pg3401_exam25/task6_141". The prompt is "prog@debian:~/pg3401_exam25/task6_141\$". The user enters the command "./main -server 127.0.0.1 -port 42147". The output shows: "Connected to 127.0.0.1:42147", "Received 809 bytes.", "Encrypted payload size: 809 bytes", "Guessing key...", "Trying key byte: 0", "Trying key byte: 1", and "Trying key byte: 2".

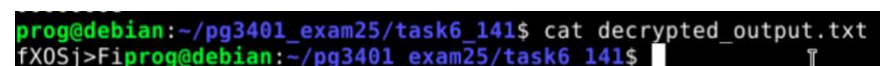
```
prog@debian:~/pg3401_exam25/task6_141$ ./main -server 127.0.0.1 -port 42147
Connected to 127.0.0.1:42147
Received 809 bytes.
Encrypted payload size: 809 bytes
Guessing key...
Trying key byte: 0
Trying key byte: 1
Trying key byte: 2
```

Figur 10: Bildet viser at programmet kobler seg til og mottar 809 bytes før den starter å gjette krypteringsnøkkelen.

A terminal window showing the continuation of the program's execution. The output continues from the previous state: "Trying key byte: 24", "Trying key byte: 25", "Found key: 19", "Decryption key found: 19191919", "Decrypted 1 blocks (8 bytes).", and "Decrypted file written to decrypted_output.txt".

```
Trying key byte: 24
Trying key byte: 25
Found key: 19
Decryption key found: 19191919
Decrypted 1 blocks (8 bytes).
Decrypted file written to decrypted_output.txt
```

Figur 11: Bildet viser at programmet finner nøkkelen, men bare dekrypterer 8 bytes.

A terminal window showing the command to view the decrypted output. The prompt is "prog@debian:~/pg3401_exam25/task6_141\$". The user enters "cat decrypted_output.txt". The output is "fX0Sj>Fi".

```
prog@debian:~/pg3401_exam25/task6_141$ cat decrypted_output.txt
fX0Sj>Fi
```

Figur 12: Bildet viser at dekryptert blokk inneholder forskjellige tegn og bokstaver.

Refleksjon:

Det er tirsdag kveld, og jeg sitter her og stirrer på koden min med trøtte øyne og en nesten tom kaffekopp ved siden av tastaturet. Jeg har nå gått gjennom samtlige funksjoner, og det slår meg hvor ukonsekvent jeg har vært med bruk av Hungarian notation og plasseringen av int og char-variabler i begynnelsen av funksjoner - noe som strengt tatt er et krav i C89-standard. Årsaken til dette er ikke manglende kunnskap, men snarere at mesteparten av tiden min har gått til intens feilsøking og debugging. Når man endelig finner en løsning som får programmet til å fungere, er det dessverre lett å glemme å gå tilbake og rydde opp i detaljer som ikke påvirker selve funksjonaliteten.

Nå, bare timer før innlevering, våger jeg rett og slett ikke å begynne å omstrukturere koden i frykt for å ødelegge det som allerede virker. Jeg har ikke kapasitet til enda en runde med mystiske bugs og "seg fault"-mareritt midt på natten. Derfor lar jeg det stå som det er - litt rotete her og der, men fullt fungerende. En slags metafor for programmering generelt, kanskje?

Denne eksamenen har vært en reise. En frustrerende, lærerik og til tider overraskende morsom reise inn i C-programmeringens verden. Jeg har sittet med hodet i hendene over manglende peker-logikk og kryptiske kompilatorfeil, bare for å juble minutter senere fordi jeg fant en manglende * i en linje jeg stirret på i tre timer. Den følelsen når programmet endelig kjører som det skal, er vanskelig å beskrive. En slags eufori som får deg til å glemme all irritasjonen, akkurat lenge nok til at du tør å kompilere en ny fil.

Et konkret eksempel: Da jeg jobbet med oppgave 4, satt jeg i flere timer og lurte på hvorfor trådene ikke samarbeidet. Jeg bannet litt (ok, mye), skrev om halve funksjonen, og vurderte å kaste laptopen ut vinduet. Men så... plutselig gikk det. Jeg hadde glemt en `pthread_join`. Den følelsen da det fungerte var nesten magisk. Det var som å løse en gåte du har tenkt på hele natten. Sliten, men lykkelig.

Gjennom dette løpet har jeg lært hvor viktig det er å jobbe strukturert - og hvor lett det er å bli dratt ned i kaoset. Jeg har fått et nytt syn på hvor kraftig, men også hvor krevende C kan være. Språket er som en motorsykel uten bremses: fryktelig raskt og effektivt - men du må vite hva du driver med.

Til slutt vil jeg si at EWA har vært en utrolig innovativ og engasjerende måte å jobbe med eksamen på. Det å få direkte tilbakemelding fra systemet, svart på hvitt om programmet faktisk fungerer, har vært både brutalt ærlig og ekstremt nyttig. Det har gitt meg muligheten til å gå tilbake og forbedre løsningene mine, og lært meg mye om å jobbe iterativt. I stedet for å levere noe halvveis og håpe på det beste, har jeg hatt sjansen til å optimalisere, teste og lære underveis. Det er verdifull erfaring jeg tar med meg videre - både som student og som fremtidig utvikler.